LAB 06

Đánh giá độ chính xác và hiệu năng

Tiny CNN V5 INT8 trên PYNQ-Z2

Slide 07 | Kiểm thử trên toàn bộ 995 ảnh MRI

Nội dung của bài học

– Chuẩn bị **995 ảnh Test** và kiểm tra số lượng theo bốn lớp.

– Đọc chương trình chạy ARM thuần và ARM + FPGA trên từng ảnh.

– Hiểu **Accuracy, Precision, Recall, F1-score** và Confusion Matrix.

– Đối chiếu **đầu ra INT8**, thời gian suy luận và giới hạn thực nghiệm.

Lab 06 khác Lab 05 ở điểm nào?

Tiêu chí	Lab 05	Lab 06
Phạm vi	Một ảnh MRI	Toàn bộ 995 ảnh Test
Số lượt	30 lượt đo / ảnh	Mỗi ảnh chạy cả hai phương án
Kết quả	Độ trễ, tensor đầu ra	Accuracy, F1, Confusion Matrix
Đích đến	Chứng minh chạy được	Đánh giá toàn bộ tập Test

Lab 06 mở rộng phạm vi đánh giá; không sử dụng lại thời gian 30 lượt của Lab 05 làm kết quả tập Test.

Mục tiêu của Lab 06

– Kiểm tra chất lượng dự đoán trên tập Test có nhãn thật.

– So sánh phương án **ARM-only** và **ARM + FPGA** bằng cùng một mô hình.

– Xác minh hai phương án cho **cùng tensor đầu ra INT8**.

– Đo thời gian suy luận và đánh giá lợi ích của bộ tăng tốc Conv1.

Tập dữ liệu kiểm thử gồm 995 ảnh MRI

Tập Test được giữ riêng từ Lab 02, không dùng để cập nhật trọng số.

Lớp MRI	Số ảnh Test
Glioma	253
Meningioma	303
No tumor	139
Pituitary	300
Tổng cộng	**995**

Khi tính Accuracy, mẫu số là 995; khi tính Recall của từng lớp, mẫu số là số ảnh thật thuộc lớp đó.

Vì sao phải dùng dữ liệu Test độc lập?

– **Training**: dữ liệu dùng để học trọng số của mạng CNN.

– **Validation**: theo dõi quá trình huấn luyện, lựa chọn mô hình.

– **Test**: kiểm tra mức độ tổng quát hóa trên dữ liệu chưa dùng để huấn luyện.

– Ở Lab 06, cả 995 ảnh Test được chạy trên PYNQ-Z2 bằng hai phương án.

Lưu ý: kiểm tra SHA-256 ở Lab 02 chỉ loại ảnh trùng hoàn toàn, chưa chứng minh độc lập theo bệnh nhân.

Chuẩn bị tập ảnh trong Jupyter

– Tệp nén: **lab06_test_995.zip**.

– Thư mục giải nén: **test_data**.

– Mỗi lớp có một thư mục con riêng.

```
test_data/
   glioma/      253 files
   meningioma/ 303 files
   no_tumor/    139 files
   pituitary/   300 files
```

Code: định nghĩa các thư mục và số lượng

```
from pathlib import Path
from zipfile import ZipFile
import shutil

ZIP_PATH = Path("lab06_test_995.zip")
DATASET_ROOT = Path("test_data")
EXPECTED = {
    "glioma": 253, "meningioma": 303,
    "no_tumor": 139, "pituitary": 300
}
```

EXPECTED được dùng để kiểm tra số ảnh từng lớp; không chỉ kiểm tra tổng 995 ảnh.

Xử lý sự khác nhau của đường dẫn Windows / Linux

– Trong một số tệp ZIP, đường dẫn ảnh lưu bằng dấu **** của Windows.

– Trên Linux, chương trình cần chuẩn hóa sang dấu **/** để tạo đúng thư mục.

– Lab 06 sửa các tệp đã giải nén nhầm tên và chỉ giải nén phần còn thiếu.

```
parts = src.name.replace("\\", "/").split("/")
label, filename = parts[-2], parts[-1]
dest = DATASET_ROOT / label / filename
dest.parent.mkdir(exist_ok=True)
src.rename(dest)
```

Code: kiểm tra đủ ảnh trước khi đánh giá

```
total = 0
for label, expected in EXPECTED.items():
    count = count_images(label)
    total += count
    print(f"{label}: {count} images")
    assert count == expected, label

assert total == 995
print("PASS: All 995 MRI test images are ready!")
```

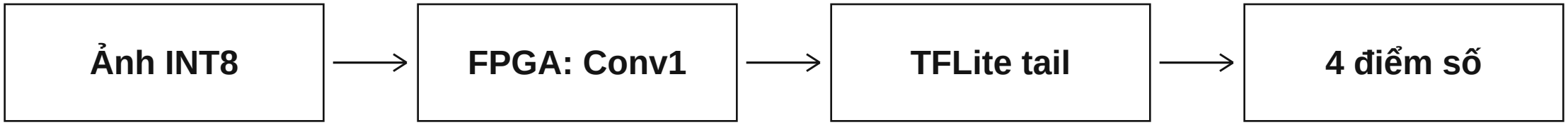
Lệnh assert chặn quá trình benchmark nếu thiếu ảnh, sai thư mục hoặc sai phân bố lớp.

Hai luồng suy luận trên cùng một ảnh

Phương án ARM thuần



Phương án ARM + FPGA



Hai mô hình và một Overlay được nạp trước

Thành phần	Vai trò
conv1_accel.bit / .hwh	Cấu hình phần cứng FPGA
tiny_cnn_v5_int8.tflite	Toàn bộ mô hình, chạy ARM thuần
tiny_cnn_v5_tail_int8.tflite	Các lớp sau MaxPool1, chạy ARM
axi_dma_0 / conv1_hls_0	Truyền dữ liệu / tăng tốc Conv1

Chỉ nạp mô hình và Overlay một lần trước vòng lặp 995 ảnh, không tính thời gian khởi tạo vào latency.

Code: khởi tạo phần cứng và Interpreter

```
ol = Overlay(str((ASSETS / "conv1_accel.bit").resolve()))
dma, conv1 = ol.axi_dma_0, ol.conv1_hls_0
full = Interpreter(model_path=str(
    ASSETS / "tiny_cnn_v5_int8.tflite"), num_threads=1)
tail = Interpreter(model_path=str(
    ASSETS / "tiny_cnn_v5_tail_int8.tflite"), num_threads=1)
full.allocate_tensors()
tail.allocate_tensors()
```

full dùng cho ARM-only; tail nhận trực tiếp tensor sau Pool1 do FPGA tính toán.

Code: bộ đệm INT8 và ranh giới FPGA – ARM

```
assert tuple(tail.get_input_details()[0]["shape"]) == (
    1, 47, 47, 8
)
ibuf = allocate(shape=(9216,), dtype=np.int8)
obuf = allocate(shape=(47, 47, 8), dtype=np.int8)
```

Tensor	Hình dạng	Dung lượng
Ảnh đầu vào	96 × 96 × 1	9.216 byte
Sau Conv1 + ReLU + Pool1	47 × 47 × 8	17.672 byte

Tiền xử lý từng ảnh MRI

```
def prepare(path):
    with Image.open(path) as img:
        img = img.convert("L").resize(
            (96, 96), Image.BILINEAR)
        pixels = np.asarray(img, dtype=np.int16)
    return (pixels - 128).astype(np.int8).reshape(
        1, 96, 96, 1)
```

– Ảnh được chuyển sang mức xám, resize song tuyến tính và lượng tử hóa INT8.

– Cùng tensor x được đưa vào cả ARM thuần và ARM + FPGA.

Code: phương án ARM thuần

```
def run_arm(x):
    t0 = time.perf_counter()
    full.set_tensor(fi, x)
    full.invoke()
    result = full.get_tensor(fo).copy()
    return result, (time.perf_counter() - t0) * 1000
```

– Đồng hồ được bắt đầu ngay trước khi đưa tensor vào Interpreter.

– Hàm trả tensor đầu ra và thời gian suy luận tính bằng mili giây.

Code: FPGA nhận ảnh qua AXI DMA

```
def run_hybrid(x):
    t0 = time.perf_counter()
    ibuf[:] = x.reshape(-1)
    ibuf.flush()
    dma.recvchannel.transfer(obuf)
    conv1.write(0x00, 1)
    dma.sendchannel.transfer(ibuf)
```

– flush() đồng bộ bộ đệm đầu vào để DMA nhìn thấy dữ liệu mới.

– DMA chuyển ảnh đến IP HLS và nhận tensor sau Pool1 từ FPGA.

Code: nhận kết quả FPGA và chạy ARM tail

```
# After DMA completion and status check
obuf.invalidate()
t1 = time.perf_counter()
pool1 = np.asarray(obuf).reshape(1, 47, 47, 8)
tail.set_tensor(ti, pool1)
tail.invoke()
result = tail.get_tensor(to).copy()
t2 = time.perf_counter()
```

– invalidate() đồng bộ dữ liệu từ DDR trước khi CPU đọc bộ đệm.

– ARM tail nhận tensor (1, 47, 47, 8) và xuất ra bốn điểm số.

Warm-up trước phép đo

– **Warm-up** là chạy thử vài lần để giảm ảnh hưởng lần khởi động đầu tiên.

– Lab 06 warm-up trên **5 ảnh mẫu**; mỗi ảnh chạy cả ARM và Hybrid.

– Đầu ra hai phương án phải khớp trước khi bắt đầu kiểm thử toàn bộ 995 ảnh.

```
for path, _ in samples[::200][:5]:
    x = prepare(path)
    a, _ = run_arm(x)
    b, _, _, _ = run_hybrid(x)
    assert np.array_equal(a, b)
```

Đánh giá đủ 995 ảnh và luân phiên thứ tự

```
for n, (path, label) in enumerate(samples, 1):
    x = prepare(path)
    if n % 2:
        a, arm_ms = run_arm(x)
        b, fpga_ms, tail_ms, hybrid_ms = run_hybrid(x)
    else:
        b, fpga_ms, tail_ms, hybrid_ms = run_hybrid(x)
        a, arm_ms = run_arm(x)
```

Luân phiên thực thi ARM → Hybrid và Hybrid → ARM để hạn chế thiên lệch do thứ tự chạy.

So sánh chính xác toàn bộ tensor INT8

```
match = bool(np.array_equal(a, b))
arm_pred = int(np.argmax(a))
hybrid_pred = int(np.argmax(b))
records.append([
    str(path), LABELS[label],
    LABELS[arm_pred], LABELS[hybrid_pred],
    int(match), arm_ms, fpga_ms, tail_ms, hybrid_ms
])
```

– array_equal so sánh **từng phần tử** tensor, chặt chẽ hơn chỉ so sánh nhãn.

– Kết quả Lab 06: **995/995 tensor đầu ra giống hệt nhau**.

Thời gian nào được tính trong benchmark?

Được tính	Không được tính
Nạp tensor vào Interpreter	Đọc ảnh MRI từ ổ đĩa
Chạy mô hình hoặc IP	Resize và chuyển ảnh thành INT8
Truyền dữ liệu bằng DMA	Nạp Overlay, tạo Interpreter
Nhận đầu ra phân loại	Chuẩn bị danh sách ảnh Test

Đây là latency của suy luận và truyền dữ liệu, không phải thời gian xử lý toàn hệ thống từ đọc ảnh đến hiển thị.

Confusion Matrix dùng để đánh giá điều gì?

– **Hàng** thể hiện nhãn thật của ảnh MRI; **cột** thể hiện nhãn dự đoán.

– Ô trên đường chéo là số ảnh dự đoán đúng cho từng lớp.

– Ô ngoài đường chéo cho biết những cặp lớp bị nhầm với nhau.

– Từ ma trận này có thể tính Accuracy, Precision, Recall và F1-score.

Accuracy: tỷ lệ dự đoán đúng

$$\text{Accuracy} = \frac{\sum_i C_{ii}}{N}$$

Trong Lab 06:

– Số ảnh đúng: **244 + 264 + 117 + 297 = 922**.

– Số ảnh kiểm thử: **995**.

– Accuracy: **922 / 995 = 92,66%**.

Precision, Recall và F1-score

$$P_i = \frac{TP_i}{TP_i + FP_i} \qquad R_i = \frac{TP_i}{TP_i + FN_i}$$

$$F1_i = \frac{2P_iR_i}{P_i + R_i} \qquad F1_{\text{macro}} = \frac{1}{4}\sum_{i=1}^{4} F1_i$$

– **Precision:** trong các ảnh dự đoán là lớp i, bao nhiêu ảnh đúng?

– **Recall:** trong các ảnh thật của lớp i, phát hiện đúng bao nhiêu ảnh?

– **Macro F1:** lấy trung bình F1 của cả bốn lớp, mỗi lớp có trọng số ngang nhau.

Code: tính các chỉ số từ Confusion Matrix

```
def metrics(cm):
    tp = np.diag(cm).astype(float)
    precision = tp / np.maximum(cm.sum(axis=0), 1)
    recall = tp / np.maximum(cm.sum(axis=1), 1)
    f1 = 2 * precision * recall / np.maximum(
        precision + recall, 1e-12)
    accuracy = tp.sum() / cm.sum()
    return accuracy, precision, recall, f1
```

Macro F1 được tính bằng trung bình số học của bốn giá trị F1 theo lớp.

Confusion Matrix trên PYNQ-Z2

Số ảnh: hàng là nhãn thật; cột là dự đoán. Hai phương án có ma trận giống nhau.

Nhãn thật / dự đoán	Glioma	Mening.	No tumor	Pituitary
Glioma	**244**	7	0	2
Meningioma	17	**264**	12	10
No tumor	3	13	**117**	6
Pituitary	1	2	0	**297**

Ví dụ: 17 ảnh Meningioma bị dự đoán thành Glioma; 13 ảnh No tumor bị dự đoán thành Meningioma.

Precision, Recall, F1-score theo từng lớp

Lớp MRI	Precision	Recall	F1-score
Glioma	0,9208	0,9644	0,9421
Meningioma	0,9231	0,8713	0,8964
No tumor	0,9070	0,8417	0,8731
Pituitary	0,9429	0,9900	0,9659

Pituitary đạt Recall cao nhất: 99,00%; No tumor thấp nhất: 84,17%.

Phân tích khả năng nhận dạng từng lớp

– **Pituitary:** dự đoán đúng 297/300 ảnh; Recall = **99,00%**.

– **Glioma:** dự đoán đúng 244/253 ảnh; Recall = **96,44%**.

– **Meningioma:** đúng 264/303 ảnh; Recall = **87,13%**.

– **No tumor:** đúng 117/139 ảnh; Recall = **84,17%**.

– Kết quả này mô tả hiệu năng trên tập Test hiện tại; không phải độ chính xác chẩn đoán bệnh nhân.

Kết quả tổng thể trên 995 ảnh

Chỉ tiêu	ARM thuần	ARM + FPGA
Dự đoán đúng / tổng	922 / 995	922 / 995
Ảnh dự đoán sai	73	73
Accuracy	92,66%	92,66%
Macro F1	0,9194	0,9194
Tensor INT8 khớp	**995 / 995**	**995 / 995**

Hai phương án không chỉ có cùng Accuracy: các tensor đầu ra INT8 cũng giống hệt trên mọi ảnh Test.

Thời gian suy luận trung vị trên PYNQ-Z2

Thành phần	Median latency
ARM thuần	23,644 ms
FPGA + DMA	1,452 ms
ARM tail	16,431 ms
ARM + FPGA	**17,855 ms**
Speedup	**1,324×**

Các trung vị được tính riêng trên 995 lần đo, không phải từ một ảnh hoặc từ 30 lượt ở Lab 05.

So sánh trực quan thời gian suy luận



Hai thanh có cùng thang đo; phần Hybrid bao gồm cả DMA, IP Conv1 và các lớp còn lại trên ARM.

Hệ số tăng tốc được tính như thế nào?

$$\text{Speedup} = \frac{T_{ARM}}{T_{Hybrid}} = \frac{23.644}{17.855} \approx 1.324$$

– Thời gian trung vị giảm **5,789 ms** so với ARM thuần.

– Hệ thống kết hợp nhanh hơn khoảng **1,324 lần** trong phạm vi phép đo.

– Không suy ra mức tiết kiệm điện năng vì Lab 06 không báo cáo phép đo công suất.

Vì sao 1,452 + 16,431 không bằng 17,855?

– FPGA + DMA, ARM tail và ARM + FPGA là **ba dãy thời gian** đo riêng.

– Lab 06 lấy **trung vị của từng dãy** sau 995 ảnh.

– **Trung vị của tổng** nhìn chung không bằng **tổng các trung vị**.

– Do đó sử dụng số **17,855 ms** được đo cho toàn bộ Hybrid để tính speedup.

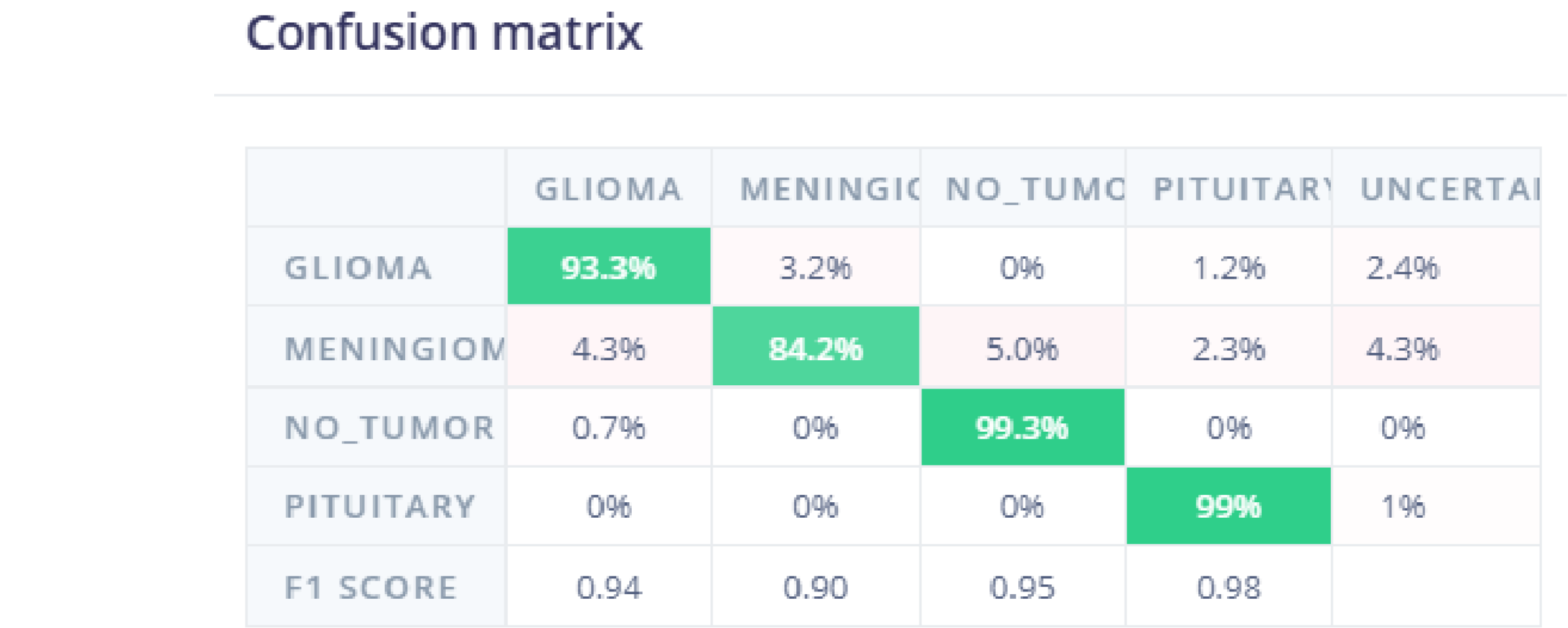
Đối chiếu với kết quả Edge Impulse

Nền tảng đánh giá	Test Accuracy INT8
Edge Impulse – Lab 03	93,07%
PYNQ-Z2 – Lab 06	92,66%
Chênh lệch	**0,41 điểm phần trăm**

– Edge Impulse có cột **UNCERTAIN**; Notebook PYNQ chọn lớp bằng **argmax**.

– Các Confusion Matrix chưa hoàn toàn cùng cách thống kê.

Minh chứng Confusion Matrix từ Edge Impulse



Ảnh giao diện từ báo cáo Lab 06: Edge Impulse có thêm cột UNCERTAIN, khác cách chọn lớp trong Notebook.

Những giới hạn phải nêu khi thuyết trình

– Edge Impulse dùng **Fit longest axis**; Notebook dùng Pillow convert("L") và resize bilinear.

– Hai pipeline tiền xử lý **chưa được chứng minh giống hệt từng pixel**.

– Recall lớp No tumor khác nhiều: **84,17%** trên PYNQ-Z2 và **99,3%** trên Edge Impulse.

– Tập Test đã lọc ảnh trùng theo SHA-256, nhưng **chưa xác minh độc lập theo bệnh nhân**.

– Không quy kết nguyên nhân của chênh lệch khi chưa có kiểm chứng bổ sung.

Các tệp kết quả được lưu sau Lab 06

Tệp	Nội dung
lab06_predictions.csv	Kết quả và thời gian theo từng ảnh
lab06_confusion_arm.csv	Ma trận nhầm lẫn ARM thuần
lab06_confusion_hybrid.csv	Ma trận nhầm lẫn ARM + FPGA
lab06_summary.txt	Accuracy, F1, median latency và speedup

Các tệp này cho phép kiểm tra lại kết quả mà không chỉ dựa vào ảnh chụp màn hình.

Kết luận Lab 06

– Trên **995 ảnh MRI**, mô hình phân loại đúng **922 ảnh**, đạt **92,66%** Accuracy.

– Macro F1 đạt **0,9194**; hai phương án cho **995/995 tensor INT8 giống nhau**.

– Median latency giảm từ **23,644 ms** còn **17,855 ms**, speedup **1,324×**.

– Tăng tốc Conv1 bằng FPGA mang lại lợi ích thực nghiệm mà vẫn giữ nguyên kết quả phân loại trên tập thử này.

Câu hỏi ôn tập cuối chương

– Vì sao phải dùng tập **Test** thay vì dựa vào Validation Accuracy?

– Một ảnh thuộc No tumor bị dự đoán thành Glioma được điền vào ô nào của Confusion Matrix?

– Vì sao **995/995 tensor INT8 khớp** là kiểm chứng mạnh hơn chỉ cùng Accuracy?

– Hệ số **1,324×** biểu thị điều gì và loại thời gian nào không được tính?

– Vì sao không thể khẳng định ngay rằng Edge Impulse và PYNQ-Z2 có tiền xử lý giống nhau?